

# 7. Relational Database Service (RDS)

---

## 1. RDS

- RDS는 **Relational Database Service** 의 약자입니다. 우리 말로 번역하면 **관계형 데이터베이스 서비스**
- AWS에서 관계형 데이터베이스를 제공해주는 서비스, **DB와 관련된 모든 작업들을 제공**
- **Fully Managed Relational Database**

## Relational Database Service (RDS)

- **완전 관리형** 관계형 DB 서비스

- **다양한 DB엔진** 제공

- ORACLE, MySQL, PostgreSQL, MariaDB, Aurora 등

- **DB 이중화 (Multi-AZ)**

- Read Replica

- 인스턴스 확장



- 완전 관리형 관계형 DB 서비스이고 ORACLE, MySQL, PostgreSQL, MariaDB, Aurora 등 다양한 DB엔진을 제공
  - **읽기 전용 복제본(Read Replica)**은 쓰기/수정이 가능한 기본 데이터베이스 (Primary/Master DB)의 데이터를 복사하여 읽기(Read) 요청만을 처리하는 데이터베이스 인스턴스입니다. 이를 통해 기본 DB의 부하를 분산하고 전체적인 데이터 처리량을 향상시킬 수 있습니다.
  - Multi-AZ의 AZ는 Availability Zone. 즉, 가용 영역을 의미
-

## 2. RDS를 사용하는 이유

- EC2 같은 서버에 직접 DB를 설치해서 사용하는 경우와 RDS를 사용하는 경우



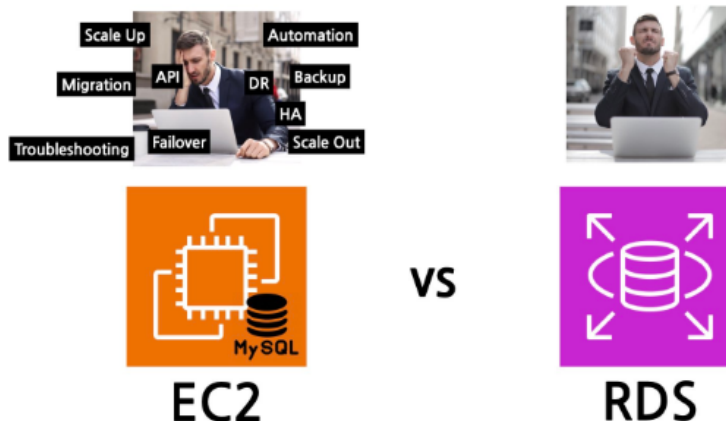
- 접 DB를 직접 관리하는 경우 해야 할 일들이 많음.

## What does a DBA do all day?

- **DBA (Database Administrator)**
- **General tasks**
  - Installation, configuration, upgrade, and migration
  - Backup and recovery
  - Database security
  - Storage and capacity planning
  - Performance monitoring and tuning
  - Troubleshooting
- **Special environments**
  - High availability
  - Very Large Databases (VLDBs)
  - Data Extraction, Transformation, and Loading (ETL)

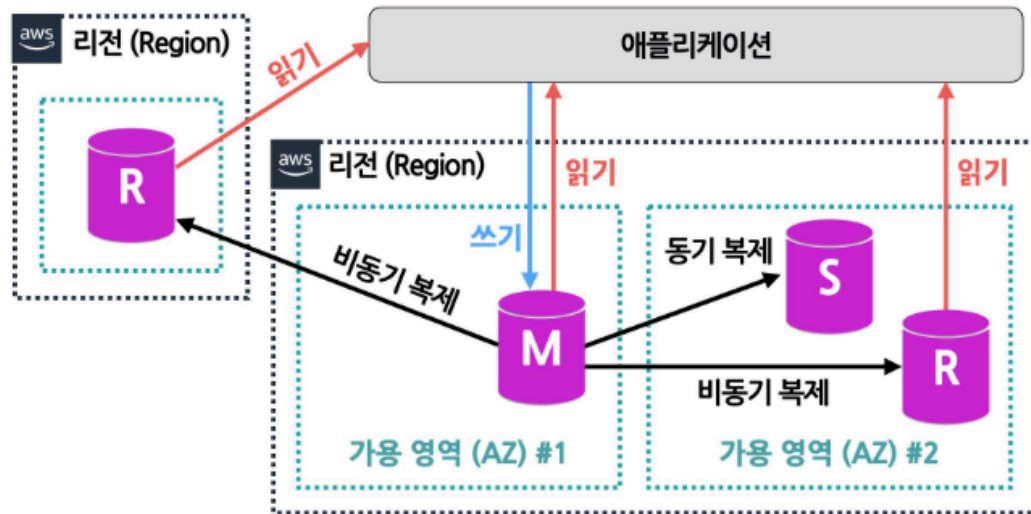
<https://www.techrepublic.com/blog/the-enterprise-cloud/what-does-a-dba-do-all-day/>

- DBA가 하루 동안 하는 일들



- 기업의 입장에서는 데이터베이스를 관리하는데 시간을 많이 쓰지 않고 다른 중요한 일들에 집중할 수 있기 때문에 편리하다.

### 3. RDS 기본 구조



- AZ-1 M 은 **Master DB 인스턴스**, Master DB를 통해 읽기와 쓰기 작업
- Master DB는 **Primary DB**라고 부름
- S라고 써있는 **Standby DB**, 데이터를 읽거나 쓰는 용도로 사용하는 것이 아닌 장애를 예방하기 위해서 **DB 이중화 작업**을 위해 사용
- Master DB의 데이터는 Standby DB로 **동기 복제**
- R이라고 써있는 인스턴스는 **Read Replica**
- Read Replica는 Master DB와 같은 리전에 있을 수도 있으며, 다른 리전에 존재할 수도 있습니다. 이처럼 다른 리전에 Read Replica를 사용함으로써, 멀리 떨어져 있는 국가에서도 빠르게 데이터를 읽을 수 있도록 합니다.
- 비동기 복제 자체가 굉장히 짧은 시간 내에 이뤄지기 때문에 거의 동일하게 데이터가 유지



- Standby DB가 Master의 역할을 하게 됩니다. 그래서 읽기와 쓰기 작업이 Standby DB를 통해 이루어진다.

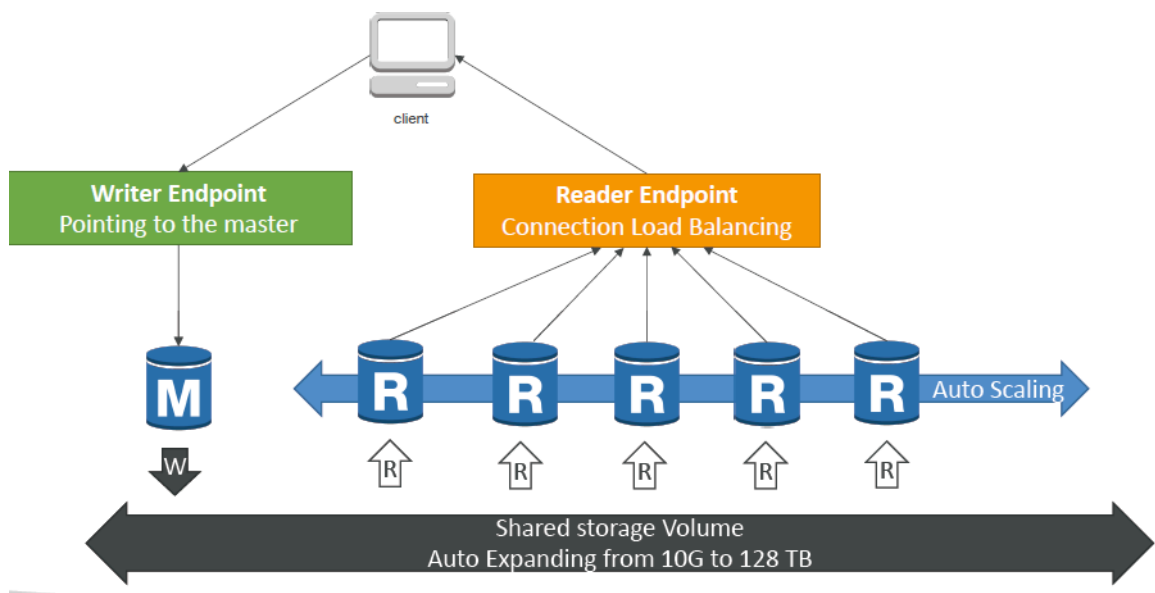
#### 다중 AZ와 읽기 전용 복제본 비교

| 다중 AZ 배포                       | 읽기 전용 복제본                        |
|--------------------------------|----------------------------------|
| 동기식 복제 - 높은 안정성                | 비동기식 복제 - 높은 확장성                 |
| 기본 인스턴스의 데이터베이스 엔진만 활성화        | 모든 읽기 전용 복제본은 접근이 가능하며 읽기 확장도 가능 |
| 자동 백업은 대기 상태에서 수행              | 기본 제공된 백업 구성 없음                  |
| 단일 지역 내에 항상 2개의 가용성 영역을 확장     | 가용 영역, 교차 AZ 또는 교차 지역 내에 있을 수 있음 |
| 기본 데이터베이스 엔진 버전 업그레이드가 발생      | 데이터베이스 엔진 버전 업그레이드는 원본 인스턴스와 독립됨 |
| 문제가 감지되면 대기 모드로 자동 Failover 조치 | 독립형 데이터베이스 인스턴스로 수동 승격 될 수 있음    |

## Amazon Aurora

- Aurora는 AWS의 독점 기술입니다(오픈 소스가 아님).
- Postgres와 MySQL 모두 Aurora DB로 지원됩니다(즉, 드라이버는 Aurora가 Postgres 또는 MySQL 데이터베이스인 것처럼 작동합니다).

- Aurora는 "AWS 클라우드에 최적화"되어 있으며 RDS의 MySQL 대비 5배, RDS의 Postgres 대비 3배 이상의 성능 향상을 제공합니다.
- Aurora 스토리지는 최대 256TB까지 10GB 단위로 자동 확장됩니다.
- Aurora는 최대 15개의 복제본을 생성할 수 있으며 복제 프로세스는 MySQL보다 빠릅니다(복제 지연 시간 10ms 미만).
- Aurora의 장애 조치는 즉각적으로 이루어지며,고가용성(HA)을 기본적으로 지원합니다.
- Aurora는 RDS보다 비용이 20% 더 높지만 효율성은 더 뛰어납니다.
- Aurora DB 클러스터

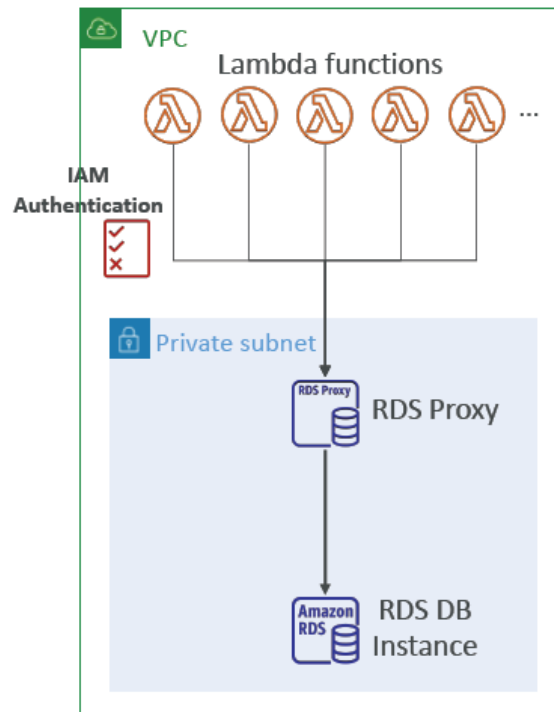


- Aurora의 주요 기능
  - 자동 장애 조치
  - 백업 및 복구
  - 격리 및 보안
  - 업계 표준 준수 Push-button scaling
  - 버튼 하나로 확장 가능
  - Zero Downtime 없는 자동 패치 적용
  - 고급 모니터링
  - 정기 유지 관리

- Backtrack: 백업 없이 언제든지 데이터 복원
- RDS & Aurora Security
  - 저장 데이터 암호화:
    - AWS KMS를 사용한 데이터베이스 마스터 및 복제본 암호화 - 시작 시 정의해야 함
    - 마스터가 암호화되지 않은 경우 읽기 복제본을 암호화할 수 없음
    - 암호화되지 않은 데이터베이스를 암호화하려면 데이터베이스 스냅샷을 생성한 후 암호화된 상태로 복원해야 함
  - 전송 중 암호화: 기본적으로 TLS 지원, 클라이언트 측에서 AWS TLS 루트 인증서 사용
  - IAM 인증: 데이터베이스에 연결하기 위한 IAM 역할(사용자 이름/암호 대신)
  - 보안 그룹: RDS/Aurora DB에 대한 네트워크 액세스 제어
  - RDS Custom을 제외하고는 SSH를 사용할 수 없음
  - 감사 로그를 활성화하고 CloudWatch Logs로 전송하여 더 오래 보존할 수 있음

## RDS Proxy

- RDS용 완전 관리형 데이터베이스 프록시
- 애플리케이션이 데이터베이스와 설정된 연결을 풀링하고 공유할 수 있도록 지원합니다.
- 데이터베이스 리소스(예: CPU, RAM)에 대한 부담을 줄이고 열린 연결(및 시간 초과)을 최소화하여 데이터베이스 효율성을 향상시킵니다.
- 서버리스, 자동 확장,고가용성(다중 가용 영역) 지원
- RDS 및 Aurora 장애 조치(fail over) 시간을 최대 66% 단축합니다.
- RDS(MySQL, PostgreSQL, MariaDB, MS SQL Server) 및 Aurora(MySQL, PostgreSQL)를 지원합니다.
- 대부분의 애플리케이션에서 코드 변경이 필요하지 않습니다.
- 데이터베이스에 대한 IAM 인증을 적용하고 자격 증명을 AWS Secrets Manager에 안전하게 저장합니다.
- RDS 프록시는 공개적으로 액세스할 수 없습니다(VPC에서만 액세스 가능).

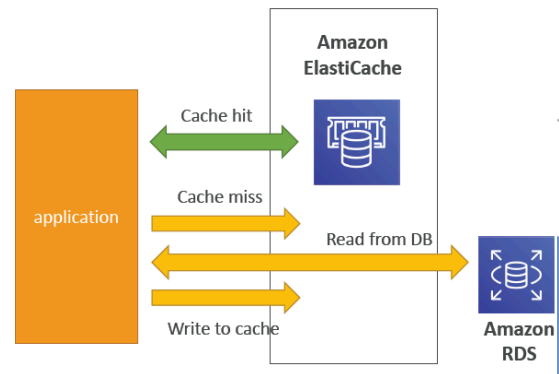


## Amazon ElastiCache

- 마이크로초 지연 시간을 제공하는 서버리스 완전 관리형 인메모리 캐싱 서비스입니다.
- 자주 사용하는 데이터를 메모리에 저장하여 데이터베이스의 부하를 줄이고 애플리케이션의 처리 속도를 획기적으로 향상
- 주요 특징 및 장점
  - **엔진 호환성:** Valkey, Redis OSS, Memcached를 완벽하게 지원합니다.
  - **완전 관리형:** 하드웨어 프로비저닝, 패치 적용, 백업, 장애 조치 등 복잡한 유지보수를 AWS가 자동 처리합니다.
  - **서버리스 옵션:** 인프라를 직접 관리하지 않고도 1분 이내에 고가용성 캐시를 생성할 수 있습니다. [1, 2, 3, 4]
  -

## ElastiCache Solution Architecture - DB Cache

- 애플리케이션은 ElastiCache에 쿼리를 보내고, 사용할 수 없는 경우 RDS에서 가져와 ElastiCache에 저장합니다.
- RDS의 부하를 줄이는 데 도움이 됩니다.
- 캐시에는 최신 데이터만 사용되도록 무효화 전략이 있어야 합니다.

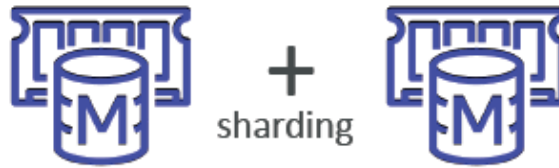


## ElastiCache이 지원하는 캐시 엔진

- **REDIS**
  - 자동 장애 조치 기능을 갖춘 멀티 가용 영역(Multi AZ)
  - 읽기 복제본을 통한 읽기 확장 및고가용성 확보
  - AOF를 사용한 데이터 내구성
  - 백업 및 복원 기능
  - 세트 및 정렬된 세트 지원



- **MEMCACHED**
  - 데이터 분할을 위한 멀티 노드 지원(분할)
  - 고가용성(복제) 미지원
  - 비영구 저장
  - 백업 및 복원(서버리스)
  - 멀티스레드 아키텍처



- 참고 : <https://aws.amazon.com/ko/elasticache/redis-vs-memcached/>

## Caching Strategies (캐싱 전략)

- ElastiCache는 애플리케이션 **성능 향상**을 위해 주로 사용된다.
- 데이터를 어떻게 저장할 것인가는 데이터의 일관성과 가용성을 유지하면서 성능을 최대화하는 데 중요하다.
- 이를 **캐싱 전략**이라고 하며 대표적인 2 가지 캐시 전략이 있다.
- **캐싱 전략**
  - **Write-Through : 바로 바로 업데이트**
    - 캐시의 데이터가 항상 **최신 상태**로 유지되므로, 데이터 일관성이 유지된다.
    - 그러나 쓰기 성능이 약간 저하될 수 있으며, 액세스되지 않는 데이터가 캐시에 쌓이는 단점이 있다
  - **Cache-Aside (Lazy loading) : 살짝 지연 but 효율적**
    - **DB에 데이터를 업데이트(write)** 우선시하고, 필요할 때만 캐시에 데이터를 업데이트한다.
    - **요청 발생**: 애플리케이션이 **데이터를 DB 요청**할 때 먼저 **캐시(ElastiCache)**를 확인한다.
      - 만약 캐시에 그 데이터가 있다면, 애플리케이션에 보낸다.
      - 만약 캐시에 그 데이터가 없다면, **데이터베이스(DB)**에서 해당 데이터를 가져온다.
        - **캐시에 저장**: 가져온 데이터를 **캐시(ElastiCache)**에 **저장**해 둔다.
        - 이후에 동일한 데이터 요청이 들어오면 캐시에서 빠르게 가져올 수 있게 됩니다.

## 4. RDS 실습

- **RDS 인스턴스 생성**

- DB 인스턴스 다중 AZ 배포로 전환
- 다른 리전에 읽기 전용 복제본 생성
- MySQL Workbench 설치 및 연결
- WordPress DB테이블 생성
- WordPress의 MySQL 연결 정보 변경
- 새로운 AMI 생성
- RDS 보안 그룹 규칙 변경
- 새로운 시작 템플릿 생성
- Auto Scaling Group의 시작 템플릿 변경
- RDS 정상 작동 테스트